

Periféricos Não Funcionais — SERPRO & Certificado Digital PJe

Data: 31/03/2026

Status: SERPRO Nacional Bloqueado por permissão de contrato | PJe Certificado Digital Bloqueado por restrição institucional

Contexto: Este documento explica **tecnicamente** e **institucionalmente** por que dois "periféricos" do Momento Fiscal não funcionam conforme esperado.

Índice

- 1. SERPRO — Endpoint Nacional de Dívida Ativa
- 2. PJe — Certificado Digital (AuthenticatePjeService)
- 3. Tabela Resumo de Bloqueios
- 4. O que precisa mudar para funcionar

1. SERPRO — Endpoint Nacional de Dívida Ativa

O que deveria funcionar

A API SERPRO de Dívida Ativa disponibiliza dois produtos distintos:

Produto	Endpoint	Cobertura
consulta-divida-ativa-df	.../consulta-divida-ativa-df/api/v1/devedor/{cpf_cnpj}	Somente Distrito Federal
consulta-divida-ativa	.../consulta-divida-ativa/api/v1/devedor/{cpf_cnpj}	Nacional (todos os estados)

A integração foi projetada para usar o endpoint nacional por padrão e cair no DF como fallback:

```
# serpro_divida_ativa_service.rb

DIVIDA_ATIVA_URL = ENV.fetch("SERPRO_DIVIDA_ATIVA_URL",
  "https://gateway.apiserpro.serpro.gov.br/consulta-divida-ativa/api/v1/devedor")

DIVIDA_ATIVA_DF_URL = "https://gateway.apiserpro.serpro.gov.br/consulta-divida-ativa-df/api/v1/devedor"
```

Por que não funciona

As credenciais atuais (`SERPRO_CONSUMER_KEY` / `SERPRO_CONSUMER_SECRET`) foram habilitadas no portal SERPRO apenas para o produto **DF**. Ao chamar o endpoint nacional, o gateway retorna **HTTP 403 Forbidden** ou **HTTP 401 Unauthorized**, pois a aplicação não tem permissão contratual para acessar o produto nacional.

O método `fetch_from_endpoint` em `serpro_divida_ativa_service.rb` trata esse cenário retornando `nil` quando recebe 403 ou 401:

```
# serpro_divida_ativa_service.rb - método fetch_from_endpoint

when Net::HTTPForbidden, Net::HTTPUnauthorized
  Rails.logger.warn("[SerproDividaAtiva] Acesso negado (#{response.code}) em #{base_url}")
  nil # ← retorna nil, sinaliza que o endpoint não está disponível
```

O método `fetch_dividas` captura esse `nil` e aciona o fallback para o endpoint DF:

```
# serpro_divida_ativa_service.rb - método fetch_dividas

resultado = fetch_from_endpoint(DIVIDA_ATIVA_URL, cnpj_limpo)
return resultado if resultado # ← só retorna se não for nil

# Nacional devolveu nil (403/401): tenta DF
Rails.logger.warn("[SerproDividaAtiva] Nacional indisponível, tentando endpoint DF...")
resultado = fetch_from_endpoint(DIVIDA_ATIVA_DF_URL, cnpj_limpo)
return resultado if resultado

[] # ← ambos falharam
```

Consequência funcional

O sistema funciona parcialmente, mas com cobertura limitada:

- Contribuintes com dívidas **no Distrito Federal** → retornados corretamente
- Contribuintes com dívidas em **SP, MG, RJ e demais estados** → retornam como "sem dívidas" (falso negativo)

Isso gera resultados inconsistentes dependendo do estado de domicílio fiscal do CPF/CNPJ consultado.

Variáveis de ambiente envolvidas

Variável	Uso
<code>SERPRO_CONSUMER_KEY</code>	Consumer Key OAuth2 da aplicação no gateway SERPRO
<code>SERPRO_CONSUMER_SECRET</code>	Consumer Secret OAuth2 da aplicação
<code>SERPRO_DIVIDA_ATIVA_URL</code>	Override do endpoint (não definida em produção = usa nacional como default, que falha)

A autenticação em si (`SerproAuthService`) funciona corretamente — o token OAuth2 é obtido com sucesso. O bloqueio acontece na chamada ao recurso, não na autenticação:

```
# serpro_auth_service.rb - autenticação funciona

AUTH_URL = "https://gateway.apiserpro.serpro.gov.br/token"

def authenticate
  request.basic_auth(ENV.fetch("SERPRO_CONSUMER_KEY"), ENV.fetch("SERPRO_CONSUMER_SECRET"))
  request.set_form_data("grant_type" => "client_credentials")
  # ← token obtido com sucesso
  # O problema está no acesso ao RECURSO, não à autenticação
end
```

Diagnóstico rápido para confirmar

```
# No servidor de produção ou localmente (com as credenciais reais):
# 1. Obter token
curl -X POST https://gateway.apiserpro.serpro.gov.br/token \
  -u "$SERPRO_CONSUMER_KEY:$SERPRO_CONSUMER_SECRET" \
  -d "grant_type=client_credentials"

# 2. Testar endpoint nacional (deve retornar 403 se não contratado)
curl -H "Authorization: Bearer <TOKEN>" \
  https://gateway.apiserpro.serpro.gov.br/consulta-divida-ativa/api/v1/devedor/60872173000121

# HTTP 403 → produto nacional não contratado (confirma o problema)
# HTTP 200 → produto nacional está habilitado (problema resolvido)
```

2. PJe — Certificado Digital (AuthenticatePjeService)

O que foi tentado

Foram testadas duas abordagens para autenticação no SSO PJe (sso.cloud.pje.jus.br/auth/realms/pje):

Abordagem A — Certificado Digital (challenge-signing via pjeoffice)

O `AuthenticatePjeService` implementa o fluxo completo de autenticação por certificado digital:

1. Acessa a página de login do SSO (`LOGIN_URL`)
2. Extrai o challenge criptográfico (parâmetro `mensagem` e `token` do URI `pjeoffice://`)
3. Assina o challenge com a chave privada do PFX usando `MD5withRSA`
4. Envia a assinatura ao SSO e obtém um `code` OAuth2
5. Troca o `code` por um JWT

```
# authenticate_pje_service.rb - fluxo call()

def call
  params = fetch_auth_params # ← extrai challenge do SSO
  signed_data = sign_pje_message(params) # ← assina com chave privada do PFX
  send_signature(params, signed_data) # ← envia assinatura
  code = authenticate_with_signature(params, signed_data) # ← obtém code OAuth2
  jwt = authorize_with_code(code) # ← troca code por JWT
  jwt
end
```

O código está correto — a implementação do fluxo é fiel ao protocolo PJeOffice. O bloqueio é externo.

Abordagem B — Usuário e Senha (Resource Owner Password Grant)

O `PjePasswordAuthService` tenta o fluxo OAuth2 com credenciais de usuário:

```
# pje_password_auth_service.rb

TOKEN_URL = "https://sso.cloud.pje.jus.br/auth/realms/pje/protocol/openid-connect/token"

req.set_form_data(
  "grant_type" => "password",
  "username" => @username, # ← PJE_USERNAME
  "password" => @password, # ← PJE_PASSWORD
  "client_id" => "portalexterno-frontend",
  "scope" => "openid"
)
```

Por que não funciona — dois bloqueios distintos

Bloqueio 1: Tipo de certificado incorreto (HTTP 500)

Os certificados e-CPF e e-CNPJ testados são do subtipo **Videoconferência (A3)**. Esses certificados são emitidos especificamente para uso em audiências telepresenciais (Portaria CNJ Nº 131/2021) e **não habilitam autenticação** na plataforma SSO do PJe.

Quando o SSO recebe a assinatura gerada por um certificado desse tipo, rejeita a sessão com **HTTP 500 Internal Server Error**, pois o perfil OID do certificado não inclui a extensão de autenticação de pessoa jurídica/magistrado esperada pelo Keycloak do PJe.

Resultado do teste documentado:

```
e-CPF Videoconferência → HTTP 500 (perfil de certificado sem permissão)
e-CNPJ Videoconferência → HTTP 500 (perfil de certificado sem permissão)
```

O certificado necessário seria do tipo **e-CPF A3 de magistrado/servidor**, emitido para uso no PJe, não para videoconferência.

Bloqueio 2: SSO PJe é de acesso institucional (HTTP 401 / negação de acesso)

O SSO Cloud (`sso.cloud.pje.jus.br`) **não é uma API pública**. O Keycloak do PJe opera em modo federado com identidades do Poder Judiciário. Requisições de:

- Empresas privadas sem convênio
- Credenciais de portais de tribunal (ex: portal TJDF) que não são federadas no SSO Cloud

...são **rejeitadas na camada de autenticação** antes mesmo de chegar ao recurso.

Resultado do teste documentado:

```
Credenciais portal TJDF → HTTP 401 (não válidas no SSO Cloud – sistemas distintos)
```

O portal `https://pje.tjdft.jus.br` usa Keycloak próprio do TJDF; o SSO Cloud (`sso.cloud.pje.jus.br`) é a instância centralizada do CNJ, com base de usuários separada.

Bloqueio 3: Convênio institucional obrigatório

Mesmo com certificado correto e credenciais válidas de um usuário do Judiciário, uma empresa privada precisa de **convênio formal** para que o SSO aceite requisições externas. Os caminhos possíveis são:

- Convênio com Tribunal parceiro (Tribunal solicita acesso ao CNJ em nome da empresa)
- Convênio direto com o CNJ (projetos de âmbito nacional)

Sem convênio, o `client_id` `portalexterno-frontend` rejeita requisições de origens não autorizadas.

Referência completa: [docs/PJE_SSO_ACESSO_INSTITUCIONAL.md](#)

Variáveis de ambiente envolvidas

Variável	Uso	Status
<code>PJE_AUTH_PFX</code>	Certificado PFX (base64) para autenticação por certificado	Tipo errado (Videoconferência)
<code>PJE_AUTH_PFX_PASSWORD</code>	Senha do arquivo PFX	—
<code>PJE_AUTH_CERTCHAIN</code>	Cadeia de certificados (JSON array)	—
<code>PJE_USERNAME</code>	Usuário para autenticação por senha	Não válido no SSO Cloud
<code>PJE_PASSWORD</code>	Senha do usuário	Não válido no SSO Cloud

Em produção as variáveis `PJE_AUTH_PFX_*` são lidas via Docker Secrets:

...

`PJE_AUTH_PFX_FILE=/run/secrets/momento_fiscal_pje_auth_pfx`

`PJE_AUTH_PFX_PASSWORD_FILE=/run/secrets/momento_fiscal_pje_auth_pfx_password`

```
PJE_AUTH_CERTCHAIN_FILE=/run/secrets/momento_fiscal_pje_auth_certchain
```

```
...
```

Token caching (GetPjeTokenService)

Quando o `AuthenticatePjeService` falha, o `GetPjeTokenService` — que é chamado pelo `PjeProcessosService` — também falha, pois depende de um token válido no cache:

```
# get_pje_token_service.rb

def call
  @token = Rails.cache.read(PJE_JWT_TOKEN_KEY) # ← cache vazio (nenhum token foi gerado)
  return @token if token_valid?                # ← false

  @token = AuthenticatePjeService.new.call      # ← levanta exceção (bloqueio no SSO)
  # ...
end
```

O resultado é que qualquer rota que dependa de `PjeProcessosService.fetch_processes_by_cpf()` vai lançar exceção antes de chegar ao endpoint PJe.

3. Tabela Resumo de Bloqueios

Periférico	Componente	HTTP Observado	Causa Raiz	Natureza do Bloqueio
SERPRO Nacional	<code>SerproDividaAtivaService#fetch_from_endpoint</code>	403 / 401	Produto <code>consulta-divida-ativa</code> nacional não habilitado no contrato	Técnica/Contratual
PJe Certificado A3	<code>AuthenticatePjeService#call</code>	500	Certificado do tipo Videoconferência não tem OID de autenticação PJe	Técnica (certificado errado)
PJe SSO Cloud	<code>AuthenticatePjeService</code> / <code>PjePasswordAuthService</code>	401 / bloqueio	SSO não é público; requer convênio institucional	Institucional
PJe Processos	<code>PjeProcessosService#fetch_processes_by_cpf</code>	Exceção Ruby	Depende de <code>GetPjeTokenService</code> que não consegue token	Cascata dos bloqueios acima

4. O que precisa mudar para funcionar

Para o SERPRO Nacional

1. Contratar o produto nacional no portal de APIs da SERPRO:

- Produto: `Consulta Divida Ativa`

- Plano que contempla todos os estados (não apenas DF)

2. Após habilitação, **nenhuma alteração de código é necessária** — o fallback já existe; basta o endpoint nacional passar a retornar 200.

3. Opcionalmente, definir `SERPRO_DIVIDA_ATIVA_URL` explicitamente no `.env` de produção apontando para o endpoint nacional para tornar a intenção explícita:

```
...
```

```
SERPRO_DIVIDA_ATIVA_URL=https://gateway.apiserpro.serpro.gov.br/consulta-divida-ativa/api/v1/devedor
```

...

Para o PJe — Certificado Digital

Opção 1 (mais rápida): Obter um certificado **e-CPF A3 para magistrado/servidor** do tipo correto, emitido para uso no PJe — não do subtipo Videoconferência. Requer usuário que seja funcionário do Judiciário.

Opção 2 (escalável): Formalizar convênio institucional com um Tribunal parceiro ou com o CNJ para obter:

- Credenciais de `service account` no SSO PJe (autenticação server-to-server)
- Permissão para o `client_id` da aplicação no realm `pje`

Após regularização:

- `AuthenticatePjeService` já implementa o fluxo correto, sem alterações de código
- `PjePasswordAuthService` já implementa OAuth2 Resource Owner Password Grant, sem alterações de código
- `GetPjeTokenService` + `PjeProcessosService` funcionarão automaticamente assim que o token for obtido

Detalhes do processo de convênio: docs/PJE_SSO_ACESSO_INSTITUCIONAL.md